

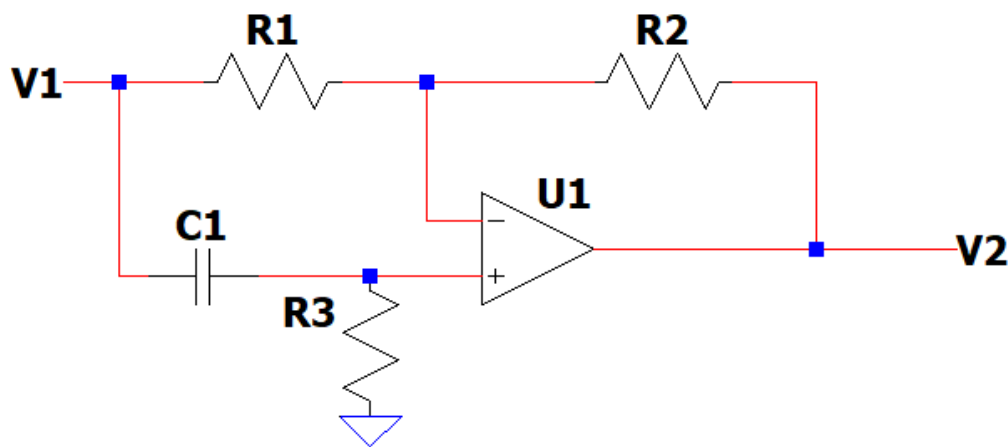
Teoría de Circuitos II

Tarea Semanal N°1

Autor: Zenobi, Matías Sebastián



Análisis del Filtro



El primer objetivo es obtener a través de un método analítico la función de transferencia del circuito.

$$T(s) = \frac{V_2(s)}{V_1(s)}$$

Consideraciones del **OPAMP** U_1 : es ideal y está correctamente retroalimentado, $V^- = V^+$.

V^- y V^+ serán los nodos a analizar para obtener la expresión de la transferencia, y su tensión se denota con $V_X(s)$.

El nodo no inversor está conectado a un divisor de tensión formado por C_1 y $R_3 \Rightarrow$

$$V_X(s) = V_1(s) \cdot \frac{R_3}{R_3 + \frac{1}{sC_1}} = V_1(s) \cdot \frac{sC_1 R_3}{sC_1 R_3 + 1} \quad (\text{I})$$

Aplico el método sistemático de nodos para el nodo inversor.

$$V_X(s) \cdot \left(\frac{1}{R_1} + \frac{1}{R_2} \right) - V_1(s) \cdot \frac{1}{R_1} - V_2(s) \cdot \frac{1}{R_2} = 0 \quad (\text{II})$$

Aplicando la ecuación (I) sobre la (II) obtenemos la expresión de la transferencia.

$$\begin{aligned} \frac{R_1 + R_2}{R_1 R_2} \cdot \frac{sC_1 R_3}{sC_1 R_3 + 1} V_1(s) - \frac{1}{R_1} V_1(s) &= \frac{1}{R_2} V_2(s) \\ R_2 \left(\frac{sC_1 R_3}{sC_1 R_3 + 1} \cdot \frac{R_1 + R_2}{R_1 R_2} - \frac{1}{R_1} \right) &= \frac{V_1(s)}{V_2(s)} = T(s) \end{aligned}$$

Desarrollando algebraicamente se llega a:

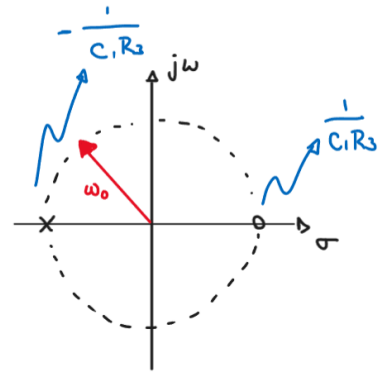
$$\begin{aligned} \cancel{R_2} \left(\frac{sC_1 R_3}{sC_1 R_3 + 1} \cdot \frac{\cancel{R_1} + R_2}{\cancel{R_1} \cancel{R_2}} - \frac{\cancel{R_2}}{\cancel{R_1}} \right) &= \frac{\cancel{C_1} \cancel{R_3}}{\cancel{C_1} \cancel{R_3}} \frac{s}{s + \frac{1}{\cancel{C_1} \cancel{R_3}}} \cdot \frac{\cancel{R_1} + R_2}{\cancel{R_1}} - \frac{\cancel{R_2}}{\cancel{R_1}} = \frac{s(R_1 + R_2) - R_2 \left(s + \frac{1}{C_1 R_3} \right)}{R_1 \left(s + \frac{1}{C_1 R_3} \right)} = \dots \\ \dots &= \frac{sR_1 + s\cancel{R_2} - s\cancel{R_2} - \frac{R_2}{C_1 R_3}}{sR_1 + \frac{R_1}{C_1 R_3}} = \frac{\cancel{R_1}}{\cancel{R_1}} \frac{s - \frac{R_2}{R_1} \frac{1}{C_1 R_3}}{s + \frac{1}{C_1 R_3}} \quad // \end{aligned}$$

$$T(s) = \frac{s - \frac{R_2}{R_1} \omega_0}{s + \omega_0}, \quad \omega_0 = \frac{1}{C_1 R_3}$$

El siguiente análisis será ver los diagramas de polos y ceros, y gráficos de módulo y fase. De ahora en más, la razón R_2/R_1 será unitaria.

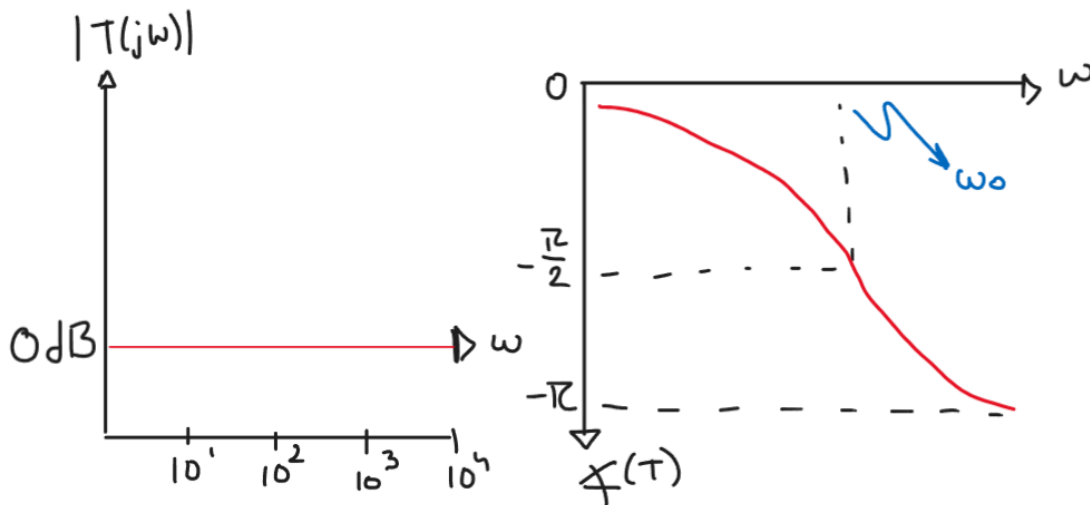
Ceros: $s_z - \omega_0 = 0 \Rightarrow s_z = \omega_0 = \frac{1}{C_1 R_3}$

Polos: $s_p + \omega_0 = 0 \Rightarrow s_p = -\omega_0 = -\frac{1}{C_1 R_3}$



Para los gráficos de módulo y fase:

$$T(j\omega) = T(s) \Big|_{s=j\omega} = \frac{-\omega_0 + j\omega}{\omega_0 + j\omega} \Rightarrow |T(j\omega)| = \frac{\sqrt{\omega_0^2 + \omega^2}}{\sqrt{\omega_0^2 + \omega^2}} = 1$$



A partir de los gráficos, se concluye que se trata de un **Filtro Pasa Todo Inversor de Fase**. Este filtro no modifica la amplitud de la señal, pero introduce un desfase dependiente de la frecuencia. Entre sus principales aplicaciones prácticas, se usa para corregir distorsiones de fase en audio y comunicaciones, así como también para introducir retardos de fase.

Normalización del circuito

Para normalizar el circuito propongo las siguientes normas:

$$\begin{cases} \Omega_Z = R_3 \Rightarrow R'_3 = \frac{R_3}{\Omega_Z} = 1 \\ \Omega_\omega = \omega_0 \Rightarrow \omega'_0 = \frac{\omega_0}{\Omega_\omega} = 1 = \frac{1}{C'_1 R'_3} \Rightarrow C'_1 = 1 \end{cases}$$

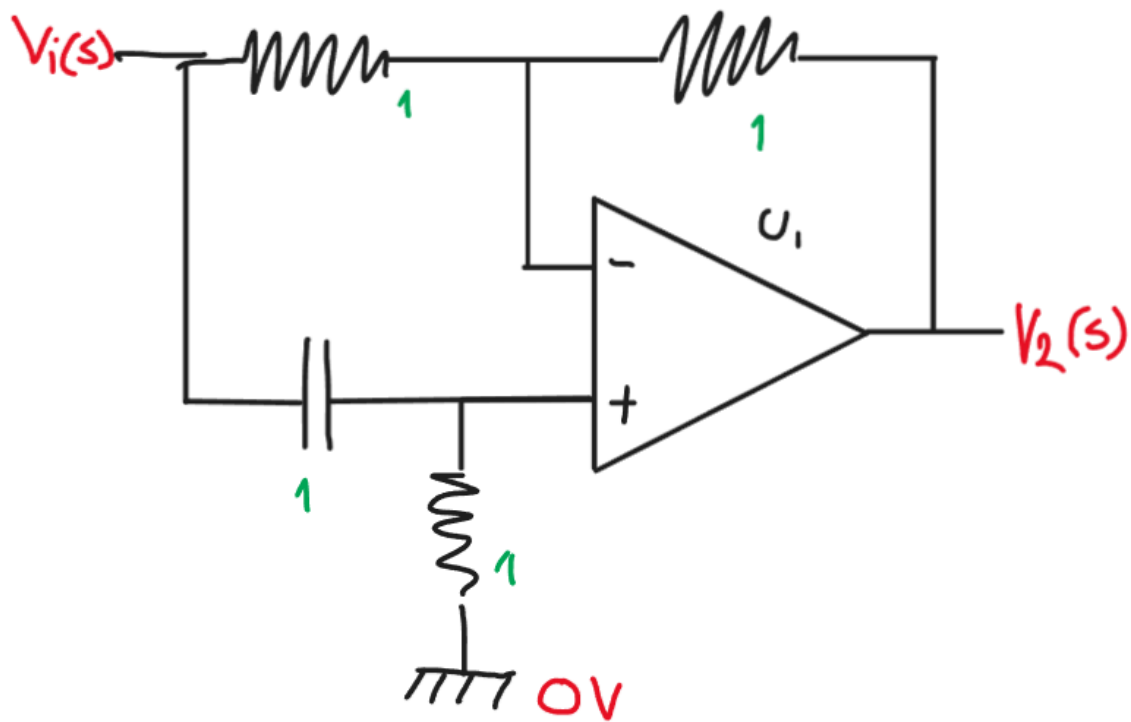
Además, considerando que la razón R_2/R_1 es unitaria, tomé la siguiente decisión de diseño:

$$R_1 = R_2 = R_3 \Rightarrow R'_1 = R'_2 = 1$$

Entonces la transferencia normalizada queda:

$$T(\$) = T(s) \Big|_{s=\frac{\$}{\Omega_\omega}} = \frac{\$ - 1}{\$ + 1}$$

El circuito que responde a la misma es:



Simulación numérica de la red a través de Python

A través del siguiente script obtuve los gráficos de módulo, fase y diagrama de polos y ceros:

```
# Módulos externos
import numpy as np
import matplotlib as mpl
from matplotlib import pyplot as plt

# Módulo de SciPy
from scipy import signal as sig

"""
# Configuración de gráficos para Jupyter
fig_sz_x = 13
fig_sz_y = 7
```

```

fig_dpi = 80 # dpi
fig_font_size = 13

mpl.rcParams['figure.figsize'] = (fig_sz_x, fig_sz_y)
mpl.rcParams['figure.dpi'] = fig_dpi
plt.rcParams.update({'font.size':fig_font_size})
"""

# Funciones de PyTC2
from pytc2.sistemas_lineales import analyze_sys, bodePlot, pz
map, pretty_print_bicuat_omegayq
from pytc2.general import print_latex, a_equal_b_latex_s

w0 = 1

num = np.array([1, -1])
den = np.array([1, 1])

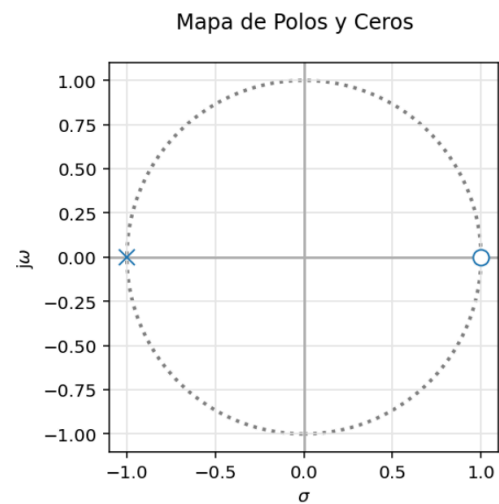
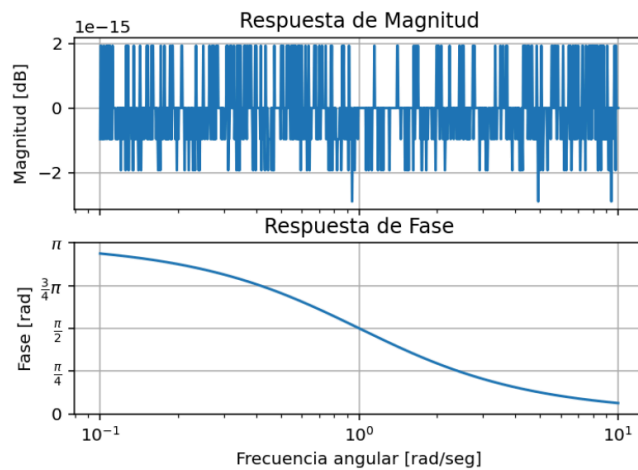
Tn = sig.TransferFunction(num, den)

_, axes_hdl = bodePlot(Tn)

plt.sca(axes_hdl[0])

pzmap(Tn)

```



Si bien la respuesta de magnitud puede aparentar un comportamiento oscilatorio, contrario a lo calculado, si observamos atentamente vemos que la amplitud es en la escala de 10^{-15} , donde es prácticamente nula.

Verificación con SymPy

Usé un script de Python que a partir de plantear dos ecuaciones con el método sistemático de nodos resuelve la ecuación, imprimiendo como resultado la transferencia en función de los componentes y la frecuencia compleja s .

```
# Imports
import sympy as sp
from pytc2.remociones import remover_polo_dc
from pytc2.general import a_equal_b_latex_s, print_latex, s,
sympfunc2tf, factorSOS
from pytc2.sistemas_lineales import bodePlot

V1, V2, Vx = sp.symbols('V1 V2 Vx') #Vx es V-=V+
Y1, Y2, Y3, YC = sp.symbols('Y1 Y2 Y3 YC')

nodo1 = sp.Equality(Vx*(Y1+Y2) - V1*Y1 - V2*Y2 , 0) # Nodo de
V-
```

```

nodo2 = sp.Equality(Vx*(YC+Y3) - V1*YC, 0) # Nodo de
V+

sol = sp.solve((nodo1, nodo2), [V2, Vx])

T_y = sp.simplify(sol[V2]/V1) #Transferencia en función de la
s admitancias

R1, R2, R3, C1 = sp.symbols('R1 R2 R3 C1')
T = sp.simplify(T_y.subs({Y1: 1/R1, Y2: 1/R2, Y3: 1/R3, YC: s
*C1}))
# Transferencia en función de los componentes y la frecuencia

print_latex(a_equal_b_latex_s('T(s) = \\frac{V_2(s)}{V_1
(s)}', T))

```

```

Python 3.12.3 (main, Mar 3 2026, 12:15:18) [GCC 13.3.0]
Type "copyright", "credits" or "license" for more information.

IPython 9.12.0 -- An enhanced Interactive Python. Type '?' for help.

In [1]: %runfile /home/zenno/.config/spyder-py3/untitled0.py --wdir


$$T(s) = \frac{V_2(s)}{V_1(s)} = \frac{C_1 R_1 R_3 s - R_2}{R_1 (C_1 R_3 s + 1)}$$


```

El resultado es una expresión equivalente a la obtenida con cálculo manual.

Para verificar la transferencia normalizada, usé un script similar donde le asigno los valores normalizados a los componentes.

```

R1, R2, R3, C1 = sp.symbols('R1 R2 R3 C1')
R1 = 1
R2 = 1
R3 = 1
C1 = 1
# T = sp.simplify(T_y.subs({Y1: 1/R1, Y2: 1/R2, Y3: 1/R3, YC:

```



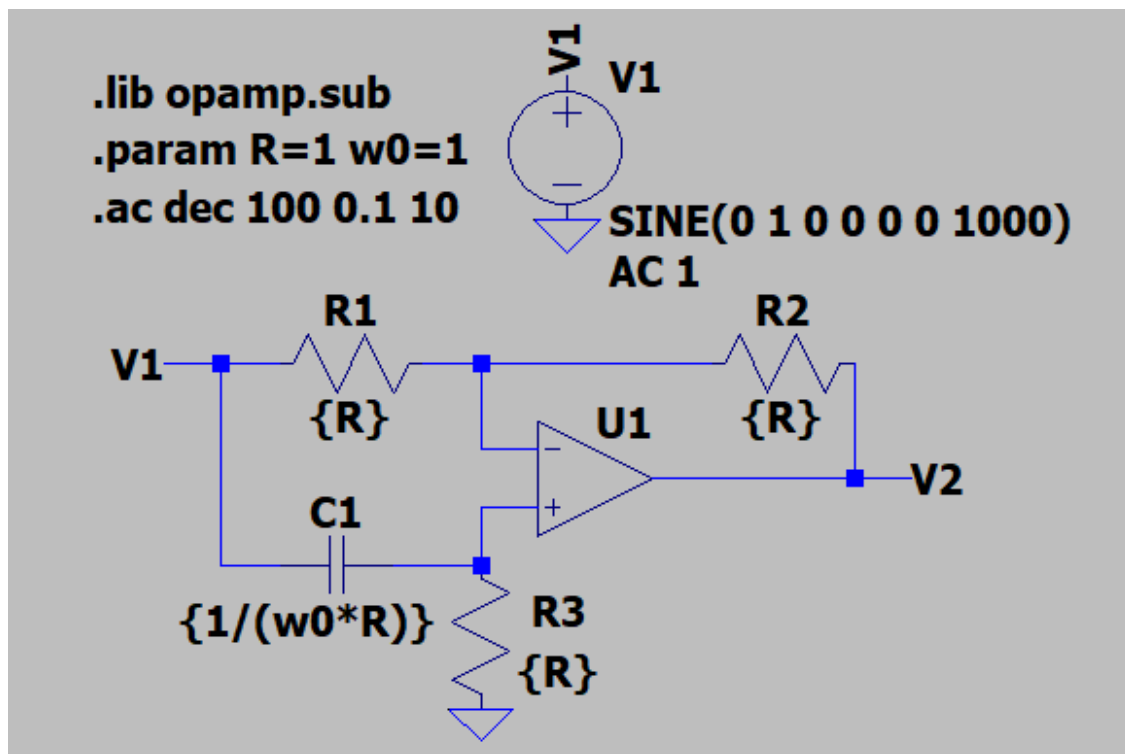
```
s*C1}))
T = sp.simplify(T_y.subs({Y1: 1/R1, Y2: 1/R2, Y3: 1/R3, YC: s
*C1}))
```

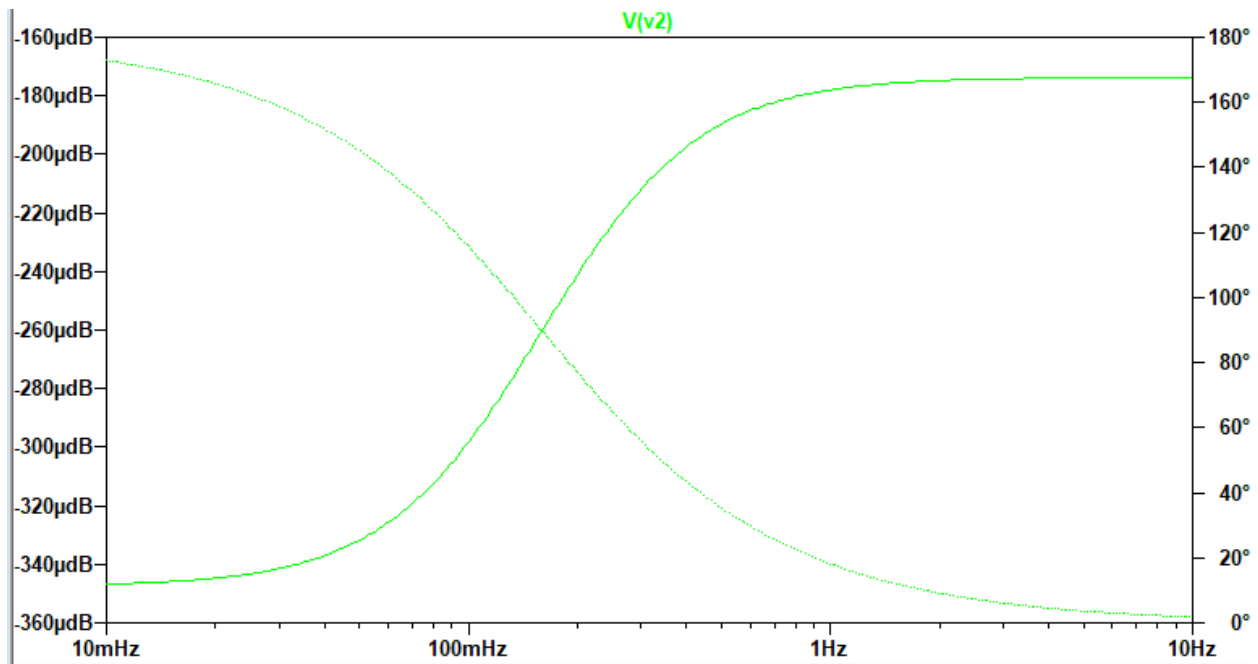
```
In [4]: %runfile /home/zenno/.config/spyder-py3/untitled0.py --wdir
```

$$T(s) = \frac{V_2(s)}{V_1(s)} = \frac{1.0(s-1)}{s+1.0}$$

Simulación en LTSpice

Construí la red normalizada y configuré un análisis de AC

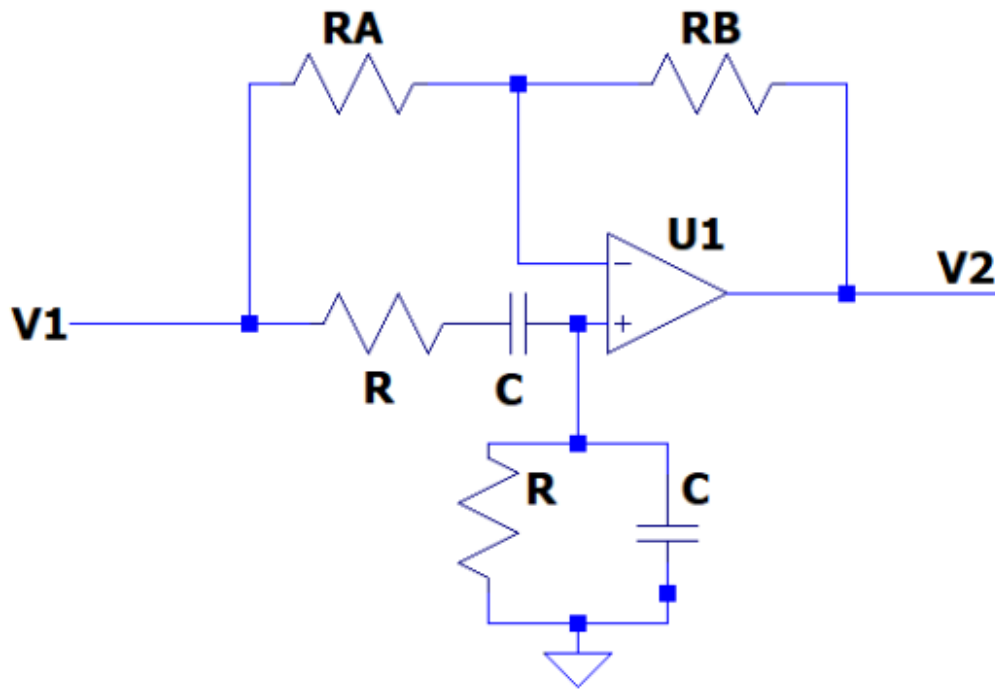




Analizando la frecuencia, el cambio de fase está situado poco después de 0.1Hz, esto es correcto ya que $\omega_0 = 1 \Rightarrow f_0 \approx 0.159\text{Hz}$.

Con respecto a la amplitud, no está fija en 0dB debido a las no idealidades del **OPAMP**, sin embargo se encuentra en torno a los μdB por lo que el error es pequeño.

Comparación con otra red



Para comparar este circuito con el Filtro Pasa Todo anterior, en primer lugar me voy a asistir de SymPy para obtener la expresión de la transferencia, que luego voy a normalizar y finalmente graficar haciendo una simulación numérica. A partir del gráfico de módulo, fase y diagrama de polos y ceros voy a analizar el filtro y concluir sus diferencias.

```
import numpy as np
import matplotlib as mpl
from matplotlib import pyplot as plt

# Módulo de SciPy
from scipy import signal as sig

V1, V2, Vx = sp.symbols('V1 V2 Vx')
YA, YB, YR, YC, YRC_s, YRC_p = sp.symbols('Y1 Y2 YR YC YRC_s YRC_p')

nodo1 = sp.Equality(Vx*(YA+YB) - V1*YA - V2*YB, 0)
nodo2 = sp.Equality(Vx*(YRC_s+YRC_p) - V1*YRC_s, 0)
```

```

sol = sp.solve((nodo1, nodo2), [V2, Vx])

T_y = sp.simplify((sol[V2]/V1).subs({YRC_s: (YR*YC)/(YR+YC),
YRC_p: YR+YC}))

RA, RB, R, C = sp.symbols('RA RB R C')
T = sp.simplify(sp.expand(T_y.subs({YA: 1/RA, YB: 1/RB, YR:
1/R, YC: s*C})))

print_latex(a_equal_b_latex_s('T(s) = \\frac{V_2(s)}{V_1(
s))', T))

```

$$T(s) = \frac{V_2(s)}{V_1(s)} = \frac{-C^2 R^2 R_B s^2 + C R R_A s - 2 C R R_B s - R_B}{R_A (C^2 R^2 s^2 + 3 C R s + 1)}$$

$$\frac{-C^2 R^2 R_B s^2 + C R R_A s - 2 C R R_B s - R_B}{R_A (C^2 R^2 s^2 + 3 C R s + 1)}$$

$$\frac{-\cancel{C^2 R^2} \cancel{R_B}}{\cancel{C^2 R^2} \cancel{R_A}} \frac{s^2 + \left(\frac{2 \cancel{C R} \cancel{R_B}}{\cancel{C^2 R^2} \cancel{R_B}} - \frac{\cancel{C R} \cancel{R_A}}{\cancel{C^2 R^2} \cancel{R_B}} \right) s - \frac{\cancel{R_B}}{\cancel{C^2 R^2} \cancel{R_B}}}{s^2 + \frac{3 \cancel{C R} s}{\cancel{C^2 R^2}} + \frac{1}{\cancel{C^2 R^2}}}$$

$$- \frac{R_B}{R_A} \cdot \frac{s^2 + \frac{2 R_B - R_A}{C R R_B} s - \frac{1}{C^2 R^2}}{s^2 + \frac{3}{C R} s + \frac{1}{C^2 R^2}}$$

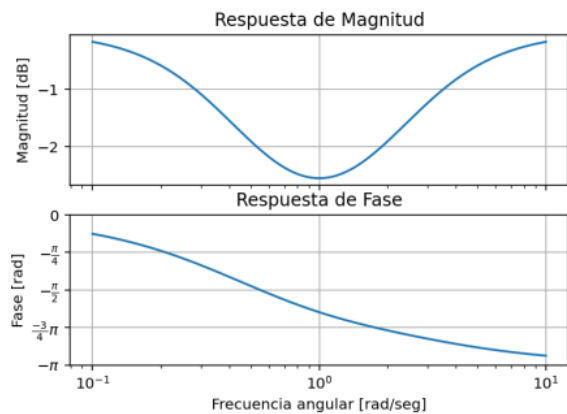
$$\frac{1}{C^2 R^2} = \omega_0^2$$

$$\omega_0' = 1 \Rightarrow C' = 1, R' = 1$$

$$\text{diseño} \Rightarrow R_A = R_B = R,$$

$$R_A' = R_B' = 1$$

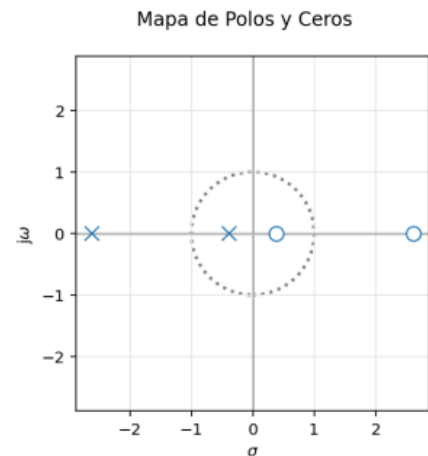
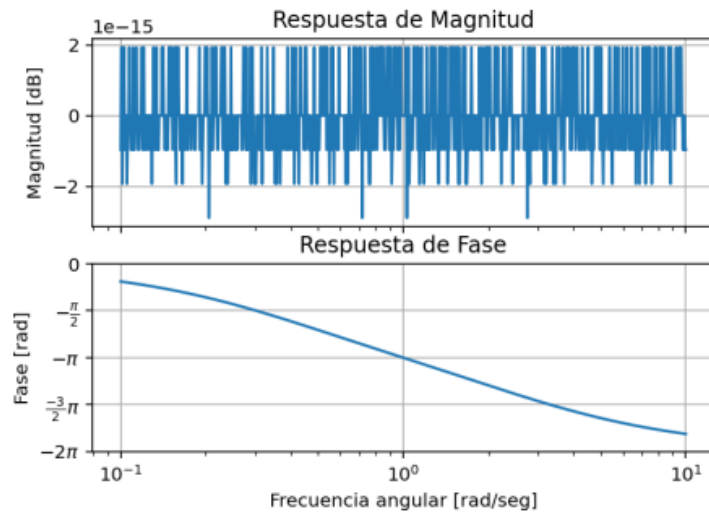
$$T_h(s) = \frac{-s^2 - s - 1}{s^2 + 3s + 1} \quad ?$$



Pese a ser un pasa todo, el gráfico me quedó como si fuera un pasa altos.

Concluyo que tuve un error a la hora de obtener la función transferencia, sé que debe existir una relación constructiva entre los parámetros R_A y R_B que iguale los coeficientes de grado 1, pero fui incapaz de corregir el error. Aún así, voy a simular el gráfico de lo que creo que debería ser la función transferencia normalizada.

$$T(s) = \frac{s^2 - 3s + 1}{s^2 + 3s + 1} \quad \text{Transferencia esperada}$$



Ahora sí son gráficos que se corresponden a un **Filtro Pasa Todo**.

Conclusión: A cambio de mayor complejidad (más componentes y es de segundo grado), este filtro permite más precisión sobre la pendiente del cambio de fase, ya

que el parámetro Q es ajustable con R_A/R_B y ofrece la posibilidad de una rotación de fase completa hasta los -360° .